INDIAN INSTITUTE OF ENGINEERING SCIENCE AND TECHNOLOGY, SHIBPUR
Howrah, West Bengal, India – 711103

DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY



A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

ON

"PROJECT TITLE"

SUBMITTED BY

Rajdeep Ghosh (510519002)
Swarnali Mondal (510510103)
Krishnendu Bera (510519004)
Arnab Sen (510519006)
Sanjana Chakravarty (510519038)
Abhiroop Mukherjee (510519109)

UNDER THE GUIDANCE OF

DR. MALAY KULE

(Academic Year: 2022–2023)

INDIAN INSTITUTE OF ENGINEERING SCIENCE AND TECHNOLOGY, SHIBPUR

Howrah, West Bengal, India – 711103

DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY



Certificate

It is certified hereby that this report, titled *whatever the title is*, and all the attached documents herewith are authentic records of Rajdeep Ghosh (510519002), Swarnali Mondal (510510103), Krishnendu Bera (510519004), Arnab Sen (510519006), Sanjana Chakravarty (510519038), and Abhiroop Mukherjee (510519109) from the Prestigious Department of Computer Science And Technology of the Distinguished and Respected IIEST Shibpur under my guidance.

The works of these students are satisfies all the requirements for which it is submitted. To the extent of my knowledge, it has not been submitted to any different institutions for the awards of degree.

Dr. Malay Kule
Asst. Professor

Dr. Asit Kumar Das
Head Of Department

ACKNOWLEDGEMENT

We, as the students of IIEST, consider ourselves honoured to be working with Dr. Malay Kule. The success of this project would not have been possible without his useful insights, appropriate guidance and necessary criticism.

We would pass our token of token of gratitude to the Department of Computer Science And Technoogy as well for providing us with the opportunity to be able to tackle real world problems while improving our problem solving ability and thinking capacity by organising this project. We all have learnt quite a handful of new skills and are eager to use them henceforth as well.

Rajdeep Ghosh (510519002)
Swarnali Mondal (510510103)
Krishnendu Bera (510519004)
Arnab Sen (510519006)
Sanjana Chakravarty (510519038)
Abhiroop Mukherjee (510519109)

Contents

1 INTRODUCTION **1**
1.1 Motivation 1
1.2 The Idea Behind The Project 1

2 PREREQUISITES **2**
2.1 Outdoor Requirements 2
2.2 Hardware and Software Requirements 2

3 THE PROJECT **2**
3.1 Software Used 2
3.2 The Program 3
3.3 The YOLO Algorithm 5

1 INTRODUCTION

1.1 Motivation

Coronaviruses are a group of related RNA viruses that cause diseases in mammals and birds. In humans and birds, they cause respiratory tract infections that can range from mild to lethal. Mild illnesses in humans include some cases of the common cold (which is also caused by other viruses, predominantly rhinoviruses), while more lethal varieties can cause SARS, MERS, and COVID-19.

With the increase in the spread of the dangerous and highly contagious **Novel Coronavirus** and the underlying disease caused by it, **COVID-19**, it is a requirement now more than ever to follow the social distancing norms set in place by the scientists and researchers.

But as we all know, India is a country with a not-so-small population, so it is pretty understandable and obvious that the law enforcement will not be able to actually enforce it on every single person. Therefore, new means of automata in place of actual individuals is a no brainier.

That is where we come in.

1.2 The Idea Behind The Project

The idea behind the working of this software was simple. The software just needed to be able to look at a live feed (or recorded footage) of a camera and know which of the people present in the footage are actually following the social distancing norms and which of them are not, and mark either one appropriately. That is where out journey to build a social distance checker started.

// will add more later probably lul

2 PREREQUISITES

2.1 Outdoor Requirements

It is important to mention here that this is not a portable software that can be fed any footage and just be expected to work. There need to be some calibration measures taken to actually get this software working:

- Actually knowing the local social distancing norms
  - The minimum distance set for social distancing by the local government
- Finding a good position for the camera
  - The footage needs to be taken from a high enough place
- Knowing the required distance in pixels
  - This will depend on the position and angle of the camera's view

2.2 Hardware and Software Requirements

The tools used to build this software are platform independent. However, there are a few requirements needed to be fulfilled to get the program working. These are:

- Software Requirements
  - Python - 3.5 or above
  - OpenCV-Python - version 2 or above
  - YOLOv3 Configuration and Network Weights
  - Numpy
- Hardware Requirements
  - A GPU is optional yet recommended to get the best performance
  - If a GPU is not being used, the CPU need to be good enough

3 THE PROJECT

3.1 Software Used

The softwares used to build this *checker* are:

3.1.1 An Integrated Development Environment (IDE)

An integrated development environment (IDE) is a software application that provides comprehensive facilities to computer programmers for software development. An IDE normally consists of at least a source code editor, build automation tools and a debugger. Some IDEs contain the necessary compiler, interpreter, or both; others, do not.

We used PyCharm as our IDE, as it was easy to set up and code

wikipedia referencePyCharm is an integrated development environment (IDE) used in computer programming, specifically for the Python language. It is developed by the Czech company JetBrains. It provides code analysis, a graphical debugger, an integrated unit tester, integration with version control systems (VCSes), and supports web development with Django as well as data science with Anaconda.

3.1.2 Python

Python is an interpreted, high-level and general-purpose programming language. Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.

Why did we choose Python?

1. Python has an upper hand when it comes to software based on image recognition and object detection. Since it is the main objective of the project, choosing python was a given.Python has an upper hand when it comes to software based on image recognition and object detection. Since it is the main objective of the project, choosing python was a given.
2. Python is unbeaten when it comes to Machine Learning. Python has support for myriad machine learning libraries, such as OpenCV, the one being used here.
3. Python is comparatively easier to understand and learn. The syntax is clear and simple to read and write.
4. And just our overall experience of using python for years.

3.1.3 Google Colab

After working on the project for quite some time, we realized that we did not have enough hardware resources at out disposal to actually make the *checker* work smoothly. So we decided on shifting to Google Colab. Google colab is an online iPython development environment similar to Jupyter Notebook. It uses CUDA acceleration to speed up processes, so we switched to it rather than continuing development

3.1.4 LaTeX

LaTeX was used to write this report. LaTeX is a software system for document preparation. When writing, the writer uses plain text as opposed to the formatted text found in "What You See Is What You Get" word processors like Microsoft Word or LibreOffice Writer.

3.2 The Program

3.2.1 Outline

The blueprint of this *checker* that we thought of initially:

1. Video Input

   Need some way to handle video input coming through the camera feed

2. Processing

   The input needs to be processed somehow

3. Detecting people

   Need to identify people in the video feed

4. Measuring distance between each couple

   Need to calculate the distance between every two persons

5. Mark the violations

   Need to mark the ones that violate social distancing norms

3.2.2 Proceedings

How we proceeded with the outlines of the blueprint:

1. Video Input

   This was easier than we expected it to be. We just had to get our hands on some recorded footage of somewhat populated areas. We refrained from using live footage because:
   - It is tough to get our hands on the light footage of a security camera or the equivalent.
   - If the checker worked on recorded footage, it would work on live footage as well.

The videos we ended up choosing:



(a) Pedestrian Video (b) Shibuya Video

Figure 1: Still Pictures from Sample Videos

2. Processing

We used the OpenCV library for our video/image processing. It is a really handy library that can be used for image processing, object detection and many other purposes.

```
import cv2 as cv

def total_frames(file_name):
    cap = cv.VideoCapture(file_name)
    res = 0

    while True:
        ret,img = cap.read()

        if not ret:
            break

        res = res+1

    return res

file_name = "pedestrian.mp4"
tot_frame = total_frames(file_name)
print(f"Total Frames in {file_name} are {tot_frame}")
```

Listing 1: A Sample Code To Count No. Of Frames in a Video

3. Detecting People

- For this we decided to go with the You Only Look Once (YOLO) algorithm for object detection. The algorithm itself is discussed a bit later in the report.
- We did not train the object detection neural network model ourselves. We used the prebuilt model, trained by the Darknet team https://pjreddie.com/darknet/yolo/ because of time constraints.

4. Measuring distance between each couple

- This was undeniably the toughest part of the project and took the longest time. First we decided to go with measuring the distance between the centroids of every two detections. But that may not work in every condition since it depends on the placement of camera and the view angle from the ground and perpendicular to the ground.
- A conversion of the 3-dimensional footage being fed to the algorithm to 2-dimensions was more than necessary to get the top view of every frame to avoid the *viewing angle problem*.
- Enter **Bird's Eye View (BEV)**. This is what we called the top view of every frame. This was made possible by OpenCV's getPerspectiveTransform() and warpPerspective() functions.

```python
def birds_eye_view(corner_points,width,height,image):
    """
    Compute the transformation matrix
    corner_points : 4 corner points selected from the image
    height, width : size of the image
    return : transformation matrix and the transformed image
    """
    # Create an array out of the 4 corner points
    corner_points = np.float32(corner_points)
    # Create an array with the parameters (the dimensions) required to build the matrix
    img_params = np.float32([[0,0],[width,0],[0,height],[width,height]])
    # Compute and return the transformation matrix
    matrix = cv.getPerspectiveTransform(corner_points,img_params)
    img_transformed = cv.warpPerspective(image,matrix,(width,height))

    return matrix,img_transformed
```

Listing 2: Function Bird's Eye Perspective Transformation Matrix

- homography estimation pdf This piece of code essentially calculates what is called a *transformation matrix* for the supplied image (frame) which can then be used to get the centroids of the points as seen from a vertical position directly above the center of the rectangle passed to the function.

```python
def birds_eye_point(matrix,centroids):
    """ Apply the perspective transformation to every ground point which have been detected
     on the main frame.
    @ matrix : the 3x3 matrix
    @ centroids : list that contains the points to transform
    return : list containing all the new points
    """
    # Compute the new coordinates of our points
    points = np.float32(centroids).reshape(-1, 1, 2)
    transformed_points = cv.perspectiveTransform(points, matrix)
    # Loop over the points and add them to the list that will be returned
    transformed_points_list = list()

    for i in range(0,transformed_points.shape[0]):
        transformed_points_list.append([transformed_points[i][0][0],transformed_points[i
    ][0][1]])

    return transformed_points_list
```

Listing 3: Function which convert co-ordinates to it's bird's view co-ordinate

- We used these functions to get a two dimensional view of every frame and calculate distance between every pair of detections (people).

5. Mark the violations

   This was again a fairly easy step. We just needed the coordinates of the people in the *violation zone* and make their detection rectangle red as opposed to green.

3.3 The YOLO Algorithm